This version gives us similar results to expect_any_instance_of, but saves us from some of its drawbacks. We've narrowed the scope to newly created instances of MetricsReporter (not including any subclasses). We can also use a pure double now, rather than a partial one.

Furthermore, this test code is more honest. The APIRequestTracker obtains its MetricsReporter instance via the new method, and our spec makes that dependency explicit. The test is awkward because the interaction in our code is awkward. Our class instantiates an object just to call one method, then discards it.

Using the Class as a Partial Double

Let's take a closer look at that short-lived reporter instance. We're not using it to track any state. We could easily avoid creating an instance by promoting the increment method to be a class method on the MetricsReporter class. Once we've updated the MetricsReporter class, we can simplify our APIRequestTracker:

15-using-test-doubles-effectively/12/api_request_tracker/lib/api_request_tracker.rb

```
class APIRequestTracker
  def process(request)
    endpoint_description = Endpoint.description_of(request)
    MetricsReporter.increment("api.requests.#{endpoint_description}")
  end
end
```

Define Class Methods Carefully



Not every method is a good candidate for moving to a class method. In particular, if you need to carry state around from one call to the next, you'll need an instance method.

If your method doesn't have any side effects, or at least doesn't use any internal state, you can safely turn it into a class method. Here, our increment method makes a call to an external service, but that's all it does. It's fine to make this a class method, and doing so will arguably improve the interface for callers.

Now that the MetricsReporter interface does not require us to create a disposable instance, we can just use the class as a partial double:

15-using-test-doubles-effectively/12/api_request_tracker/spec/api_request_tracker_spec.rb

```
it 'increments the request counter' do
  expect(MetricsReporter).to receive(:increment).with(
    'api.requests.get_users'
  )

  APIRequestTracker.new.process(request)
end
```